

Experiment -3 Report.

EE19B108 -BATCH-1

Objectives:

To understand how to access peripherals using the Atmel Atmega8 microcontroller and the concept of Memory Mapped Input-Output. To achieve this we are building:

- AVR assembly program to blink an LED with a pulse width of 0.5 sec and pulse repetition rate of 1 cycle per second.
- AVR assembly program to control an LED with a push-button.
- 4-bit nibble addition for an input from an 8-bit DIP switch.

Hardware and Software Used:

- Atmel8L
- Breadboard
- 8-bit DIP switches
- 5 LEDs
- Resistors and wires
- AVR Programmer
- Microchip Studio IDE

Working with Peripherals in AVR Assembly Programming:

AVR microcontrollers, like the Atmega8, use a method called Memory-mapped I/O. This means that both memory and I/O (input/output) registers are accessed through the same address space.

The Atmega8 microcontroller has a total of 23 I/O pins, which are organized into three separate ports:

- **PORTB**
- **PORTC**
- **PORTD**

Each of these ports can be accessed using specific memory addresses.

To work with these ports, you interact with three primary types of registers:

1. **Data Direction Registers (DDRs):** These registers set each pin of the corresponding port as either an input or an output.
2. **PORT Registers:** These registers are used to send signals out to the pins.
3. **PIN Registers:** These registers are used to read signals coming from the pins.

To configure the pins, you use the **OUT** instruction to write a value to the appropriate DDR register. Each bit in the DDR register determines whether a pin is set to input (0) or output (1).

You can't directly use immediate values like **0xFF** to set DDR values. Instead, you need to load these values into temporary registers first, then use these temporary registers to set the DDR values.

To read data from a pin, you use the **IN** instruction. You specify the destination register where the data will be stored as the first operand, and the source PIN register (such as PINB, PINC, or PIND) as the second operand.

For sending data to the pins, you use the **OUT** instruction. You specify the PORT register (such as PORTB, PORTC, or PORTD) as the first operand, and the register containing the data to be sent as the second operand.

Challenges with Handling Input Pins:

- When input pins are left unconnected and not tied to a specific voltage source, they enter a state known as high-impedance or floating. In this floating state, the voltage on the pin can become unpredictable and is affected by several factors.
- These factors include the pin's capacitance, which can retain a charge and produce a certain voltage even without an external voltage source. Leakage currents can also introduce unintended voltages, and even touching the pins can change the voltage due to static electricity.
- This unpredictability can cause issues when input pins are left floating, resulting in unreliable or incorrect readings.
- To solve this problem, pull-up or pull-down resistors are employed. These resistors establish a default voltage level for the pins, setting them to high or low, respectively, when no active voltage source is connected.

Programming the Atmega8:

- The assembly code is first built to create a .hex file.
- The AVR programmer is then used to connect the PC to the microcontroller, the connections to be made are as given below.

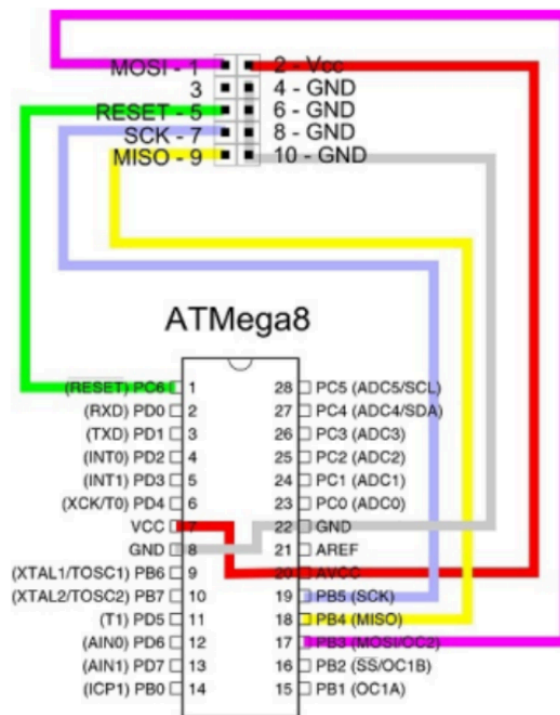


Figure 1: Connections from AVR programmer to the microcontroller

- Then, open AVR Burn-O-Mat, select the hex file and click on write to program the microcontroller.

Implementation of a blinking LED:

1. Code for the blinking LED:

```
.CSEG          ; Start of code segment
LDI R20, 0x02  ; Load immediate value 0x02 into register R20
OUT DDRB, R20  ; Set data direction for PORTB to output (0x02 sets
PB1 as output)

again:
LDI R20, 0x00  ; Load immediate value 0x00 into register R20
OUT PORTB, R20 ; Set PORTB to 0x00 (turn off all bits on PORTB)

; Delay loop
```

```

LDI R16, 0xFF ; Load immediate value 0xFF into register R16 (outer
loop counter)
loop2:
LDI R17, 0xFF ; Load immediate value 0xFF into register R17 (inner
loop counter)
loop3:
DEC R17 ; Decrement R17
BRNE loop3 ; If R17 is not zero, repeat inner loop
DEC R16 ; Decrement R16
BRNE loop2 ; If R16 is not zero, repeat outer loop

; Turn on bit in PORTB
LDI R20, 0x02 ; Load immediate value 0x02 into register R20
OUT PORTB, R20 ; Set PORTB to 0x02 (turn on bit corresponding to
0x02, e.g., PB1)

; Delay loop
LDI R16, 0xFF ; Load immediate value 0xFF into register R16 (outer
loop counter)
loop4:
LDI R17, 0xFF ; Load immediate value 0xFF into register R17 (inner
loop counter)
loop5:
DEC R17 ; Decrement R17
BRNE loop5 ; If R17 is not zero, repeat inner loop
DEC R16 ; Decrement R16
BRNE loop4 ; If R16 is not zero, repeat outer loop

; Turn on bit in PORTB again
LDI R20, 0x02 ; Load immediate value 0x02 into register R20
OUT PORTB, R20 ; Set PORTB to 0x02 (turn on bit corresponding to
0x02, e.g., PB1)

```

Listing 1: Code for the blinking LED

2. Input and output:

- The program doesn't take any input.
- The output is an LED that blinks at the frequency of 1Hz and a duty cycle of 50

3. Blinking LED Operation:

- To make an LED blink, you need a delay routine of 0.5 seconds.
- The Atmega8 microcontroller operates with a default clock frequency of 1MHz. To create a 0.5-second delay, you need 500,000 clock cycles.
- This delay is achieved through a 3-tiered loop structure: the outer loop iterates 5 times, the middle loop runs 100 times with each iteration causing a 5-clock cycle delay, and the innermost loop executes 2 times with a 5-clock cycle delay per iteration. Together, these loops provide a delay close to 500,000 clock cycles.
- To set PORTD as an output, you load DDRD with the value 0xFF.
- Register R16 holds the value that determines the LED's state. Initially, R16 is set to 0x00. After executing the delay loop, R16's value is inverted (using the 1's complement), and this new value is written to PORTD.
- Each time the value is inverted, the LED state on PORTD is updated using the command "OUT PORTD, R16." The LED is connected to pin D0 and is grounded through a resistor.
- This arrangement results in the LED blinking at a frequency of 1Hz, with a 50% duty cycle.

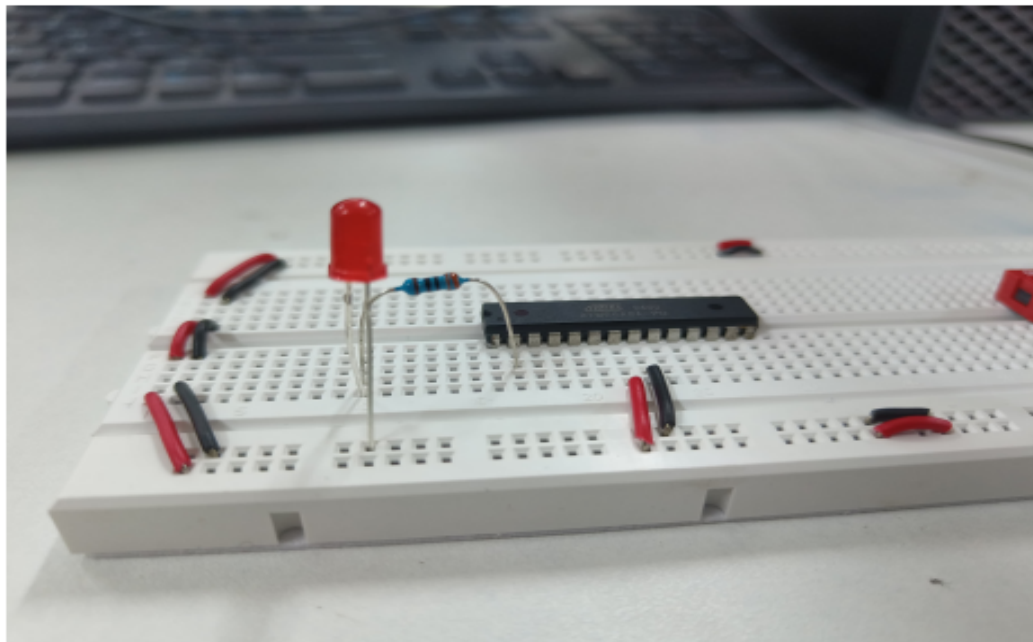


Figure 2: LED turned off

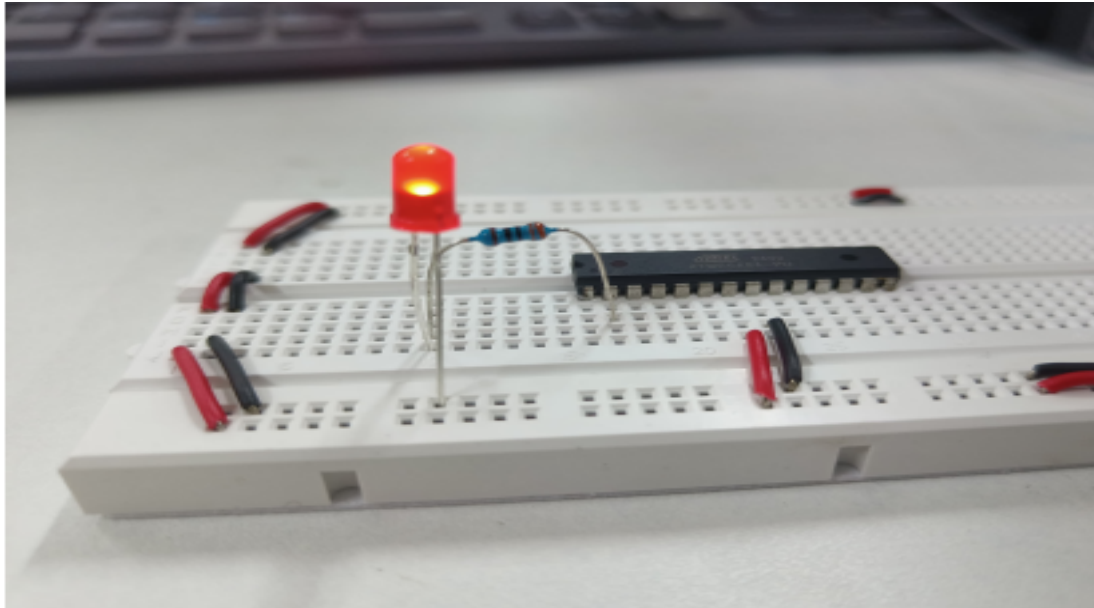


Figure 3: LED turned on

- We can also change the clock frequency by changing the CKSEL fuse Byte while programming.
- The above images show the LED in both on and off states.

Implementation of the LED controller:

1. Code for the LED controller:

```
.CSEG           ; Start of code segment
```

```
LDI R25, 0x00   ; Load immediate value 0x00 into register R25  
(used for DDRC)
```

```
LDI R26, 0xFF ; Load immediate value 0xFF into register R26
               (used for DDRD)
```

```
OUT DDRD, R26 ; Set all pins of PORTD as output (0xFF means
all bits are set for output)
```

```
OUT DDRC, R25 ; Set all pins of PORTC as input (0x00 means
all bits are set for input)
```

```
loop:
```

```
IN R16, PINC ; Read the value of PORTC into register R16
              (since PORTC is set as input, PINC holds the value of PORTC)
OUT PORTD, R16 ; Output the value from R16 to PORTD (display
the value read from PORTC to PORTD)
```

```
RJMP loop ; Relative jump to 'loop' label (creates an
infinite loop)
```

Listing 2: Code for the LED controller

2. Input and output:

- The program takes in input via a push-button through PORTB.
- The output is an LED that turns on when the push button is pressed and is off when the push button isn't pressed.

3. Operation of the LED Controller:

- The program runs in an infinite loop, continuously reading input from PINB and sending it as output to PORTD.
- PORTB is configured as an input by setting DDRB to 0x00, while PORTD is configured as an output by setting DDRD to 0xFF.
- A push button is connected to pin C0 through a pull-down resistor to avoid a high-impedance state, and the LED is connected to pin D0.
- At the beginning of each loop iteration, the value from PINB is loaded into register R16 using the "IN R16, PINB" instruction.
- The value in R16 is then written to PORTD using the "OUT PORTD, R16" instruction.

- When the least significant bit (LSB) of PORTB is 1 (indicating the push button is pressed), the LED turns on. When the LSB is 0, the LED turns off.

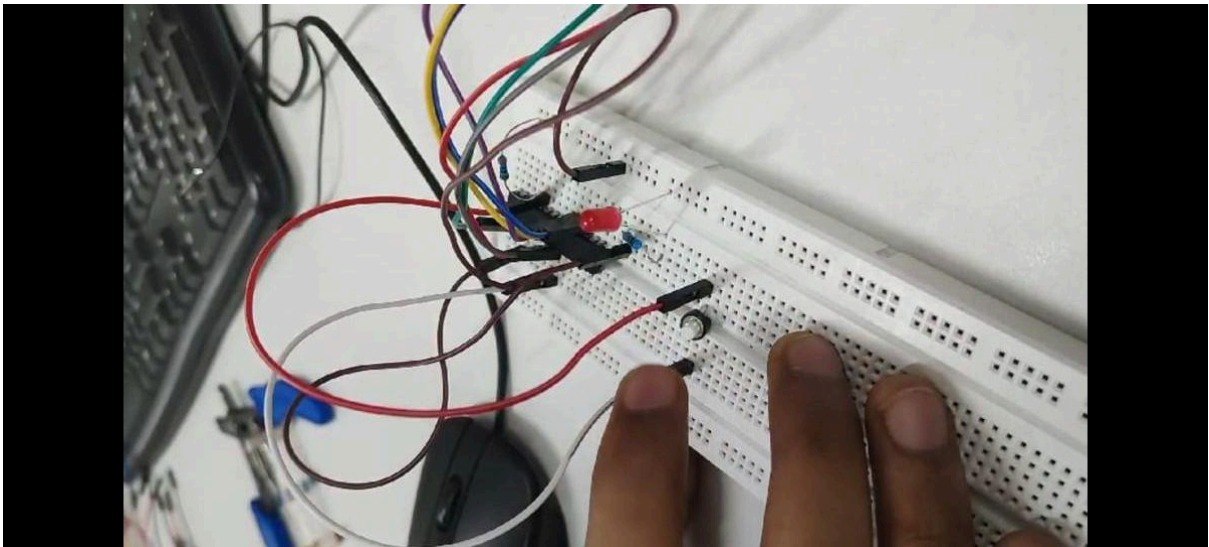


Figure shows the led when push button is not pressed

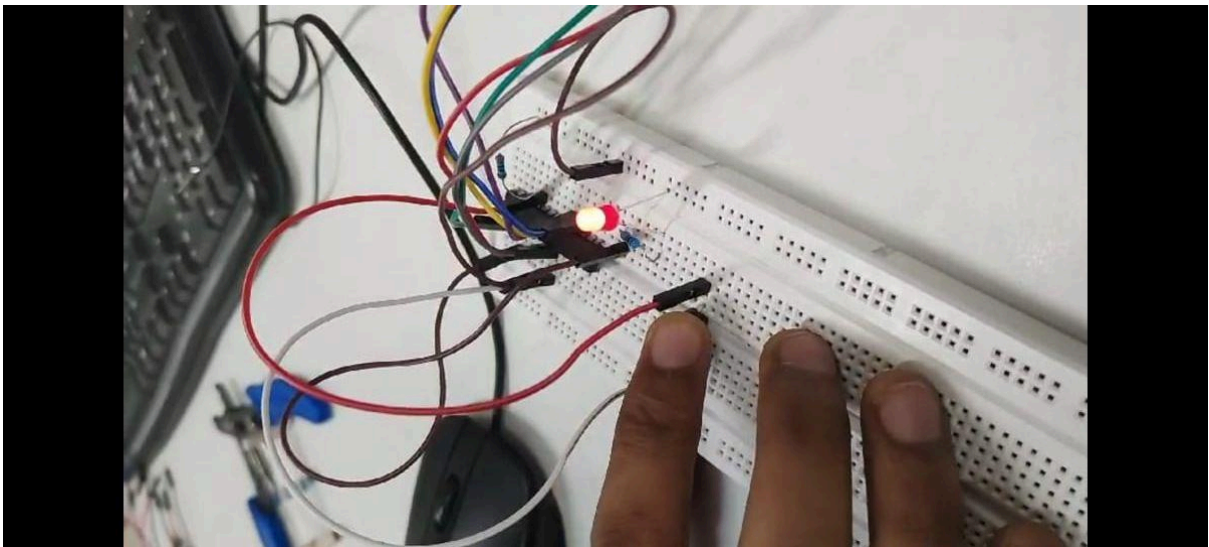


Figure shows the led when push button is pressed

Implementation of the 4-bit adder:

1. Code for the 4-bit adder:

```
.CSEG           ; Start of code segment
```


LDI R25, 0x00 ; Load immediate value 0x00 into register R25 (used to set DDRD)

LDI R26, 0xFF ; Load immediate value 0xFF into register R26 (used to set DDRC)

OUT DDRD, R25 ; Set all pins of PORTD as input (0x00 means all bits are set for input)

OUT DDRC, R26 ; Set all pins of PORTC as output (0xFF means all bits are set for output)

LDI R18, 0x0F ; Load immediate value 0x0F into register R18 (used as a mask to isolate lower nibble)

LDI R19, 0xF0 ; Load immediate value 0xF0 into register R19 (used as a mask to isolate upper nibble)

loop:

IN R16, PIND ; Read the value of PORTD into register R16 (since DDRD is set as input, PIND holds the value of PORTD)

MOV R17, R16 ; Copy the value of R16 to R17 for further processing

AND R16, R18 ; Perform bitwise AND operation between R16 and R18 (isolates lower nibble of R16)

AND R17, R19 ; Perform bitwise AND operation between R17 and R19 (isolates upper nibble of R17)

LSR R17 ; Logical Shift Right R17 (shift bits to the right by one position)

LSR R17 ; Logical Shift Right R17 (shift bits to the right by one position)

LSR R17 ; Logical Shift Right R17 (shift bits to the right by one position)

LSR R17 ; Logical Shift Right R17 (shift bits to the right by one position)

ADD R16, R17 ; Add the value of R17 to R16 (combines the processed upper nibble with the lower nibble)

OUT PORTC, R16 ; Output the result in R16 to PORTC (since DDRC is set as output, this sends data to PORTC)

RJMP loop ; Relative jump to 'loop' label (creates an infinite loop)

Listing 3: Code for the 4-bit adder

2. Input and output:

- The program takes input from an 8-bit DIP switch through PORTD. The first four bits are the first number to be added and the last 4 bits are the second number to be added.
- The output is the sum of the two 4-bit numbers displayed using 5-LEDs via PORTC 3. Working of the 4-bit adder
- The program is an infinite loop that takes in the two nibbles that are to be added via PIND, adds them and writes the output to PORTC which are connected to 5 LEDs that display the output sum.
- The input is given through a DIP switch in the format mentioned in the previous section. PIND is used to take in the input and is configured by setting the register DDRD to 0x00.
- Each input pin of PIND is connected to the ground via a pull-down resistor to ensure we don't get a high-impedance state at the pin. The same pins are also connected to the dip switch to get the input.
- The input is loaded onto R16 by using the instruction "IN R16, PIND".
- R17 is loaded with 0xf0, an and operation is performed using the instruction "AND R17, R16" to isolate and store the first nibble in R1. The operation "LSR R17" is also performed 4 times to correctly transfer the nibble to R17.
- The other nibble is isolated and stored in R16 by performing the and operation with 0x0f using the instruction "ANDI R16, 0x0f".
- These isolated nibbles are added using the instruction "ADD R16, R17" which adds both the numbers and stores the result in R16. • Finally the sum in R16 is written to the PORTC as output using the instruction "OUT PORTC, R16".

- The pins of PORTC are connected to LEDs whose other terminal is connected to the ground via resistors to limit the current through the LED. • Below are annotated images that show the inputs and the corresponding outputs taken after experimenting.

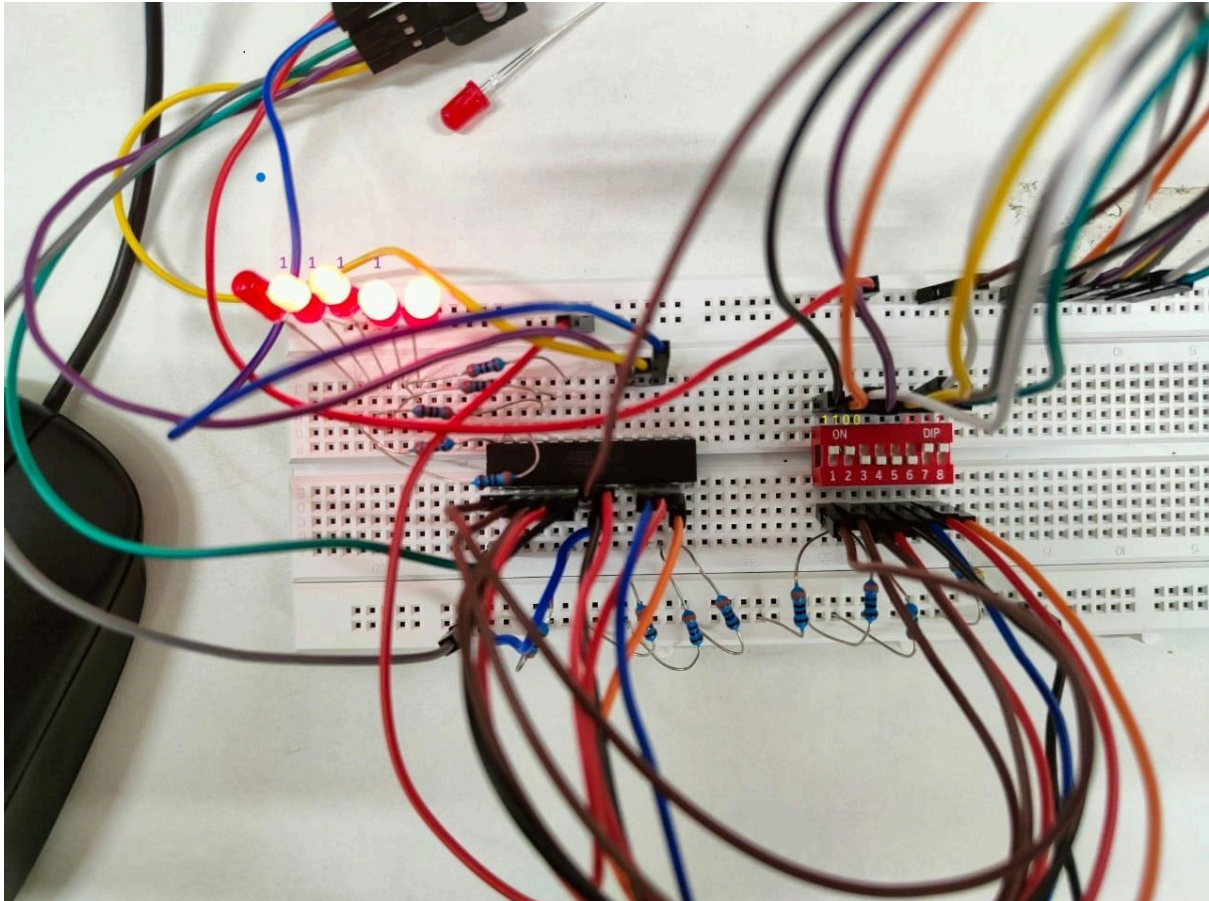


Figure shows the sum of 2 4-bit numbers

Inferences and Conclusion:

- The Atmega8 microcontroller, like other AVR-based devices, uses address-mapped I/O, which simplifies interaction with peripherals by allowing standard memory operations (such as LDI, LPM, and ST) to be used for peripheral access. This integration streamlines peripheral management but complicates address management because both memory and I/O registers share the same address space. Address-mapped I/O devices generally offer faster access compared to port-mapped devices, like those in the x86 family.

- Through this process, we learned how to configure I/O ports and perform I/O tasks using various I/O registers. Additionally, we explored how to implement delays in AVR assembly programs. We also discovered that the internal clock frequency can be adjusted by modifying the CKSEL fuse bits, with the default set at 1 MHz.
- To program an Atmega8 or similar microcontroller, the code must first be compiled into a hex file. This file is then uploaded to the microcontroller using an AVR programmer and software such as AVR Burn-O-Mat. We also addressed the issue of high-impedance states by using pull-up or pull-down resistors to ensure that inputs are read correctly.

----- **End**-----